BBM-382 – SOFTWARE ENGINEERING

SPRING 2025

Lecture 13

Prof. Dr. Ayça KOLUKISA
Assoc. Prof. Dr. Ebru GÖKALP-AYDIN
Asst. Prof. Dr. Tuğba ERDOĞAN

CHAPTER 24 – QUALITY MANAGEMENT

Topics covered

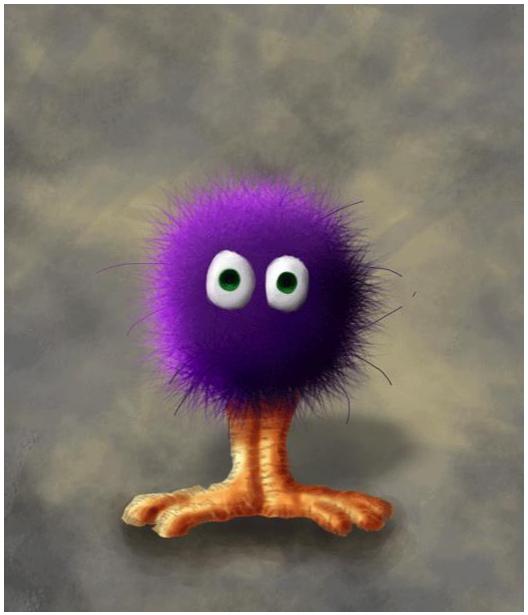
- Software quality
- Software standards
- Reviews and inspections
- Software measurement and metrics

Software Quality Management

- What is Software Quality?

- How is Software Quality managed?

What is Sofware Quality?



How is Software Quality managed?

Quality of Measurement Results

Software Enginnering Culture & Ethics

Software Process Management

Process Automation

Software Products Measurement

Software Quality Assurance

Software Quality Management Techniques

Process Assessment Methods

Process Measurement Techniques

Quality Improvement

Reviews and Audits

Verification & Validation

Software Life Cycle Models

Process Infrastucture

Process Assessment Models

Value and Cost of Quality

Notations for Process Definition

Software Quality Requirements

Software Life Cycle Processes

Software Quality Measurement

Process Adaptation

Defect Characterization

Models for Process Implementation and Change

Process Measurement

Models and Quality Characteritics

Software Information Models

SQM Scope (Reference: SWEBOK v2)

Quality of Professional Software

- When we talk about the quality of professional software, we have to take into account that the software is used and changed by people apart from its developers.
  - Quality is therefore not just concerned with what the software does; it has to include the software's behavior while it is executing and the structure and organization of the system programs and associated documentation.
- This is so-called quality or non-functional software attributes.
  - Examples of these attributes are the *software's response time to a user query* and the *understandability of the program code*.
  - The specific set of attributes that you might expect from a software system obviously depends on its application domain.
    - E.g., a banking system must be secure, an interactive game must be responsive, a telephone switching system must be reliable, and so on.

Quality – Some Definitions

- The word "quality" is derived from the latin "qualitas", meaning "of what"
  - Its use goes back to antiquity
  - The morfem "-tas" was used to create "nature" or "character"

- Conformance to requirements (Crosby, 1979)
  - Producer perspective

- Fitness for use (Juran, 1951)
  - Customer-oriented view

- The degree to which a set of inherent characteristics fulfills the requirements, i.e. needs or expectations that are stated, generally implied or obligatory. (ISO 9000)

Software Quality is a bit Different (Gillies)

- Software has no physical existence
- The lack of knowledge of client needs at the start
- The change of client needs over time
- The rapid rate of change in both hardware and software
- The high expectations of customers, particularly with respect to adaptability
- No distinction between ‘design quality’ and ‘manufacturing quality’

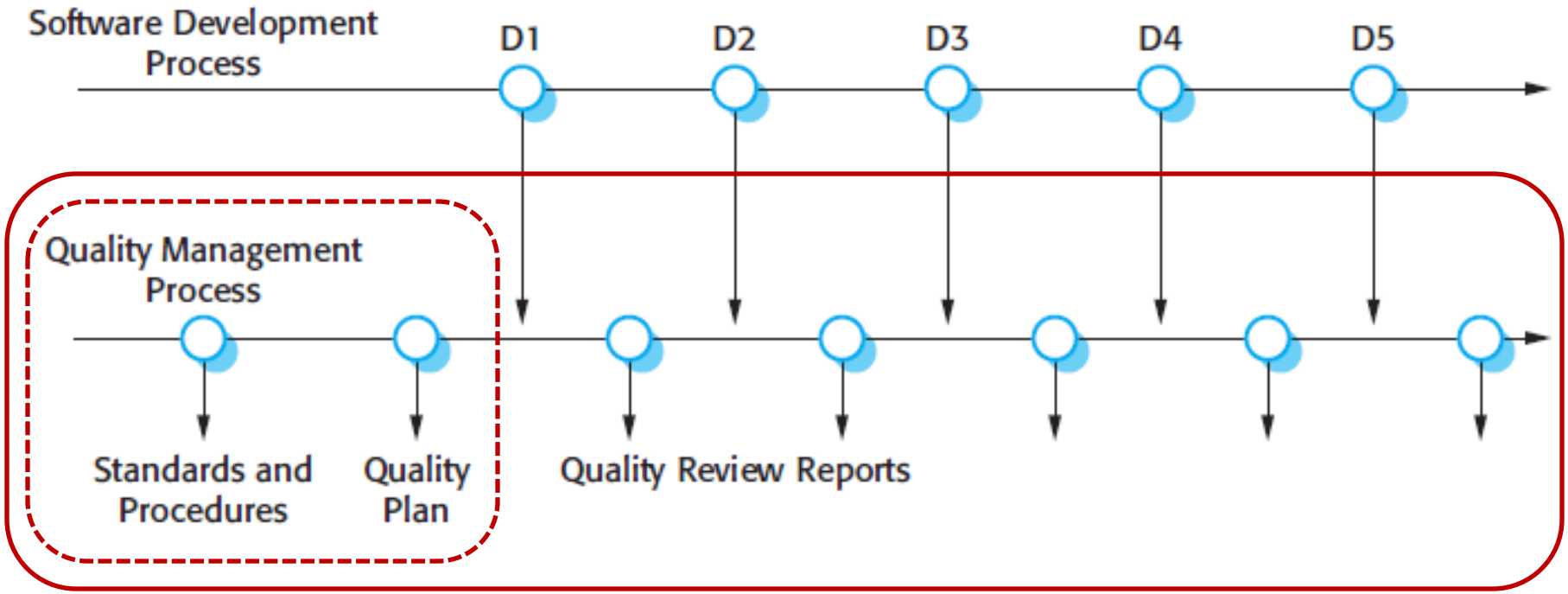
Software quality management

- Concerned with ensuring that the required level of quality is achieved in a software product.

- Quality management provides an independent check on the software development process.
  - It checks project deliverables to ensure they are consistent with organizational standards and goals

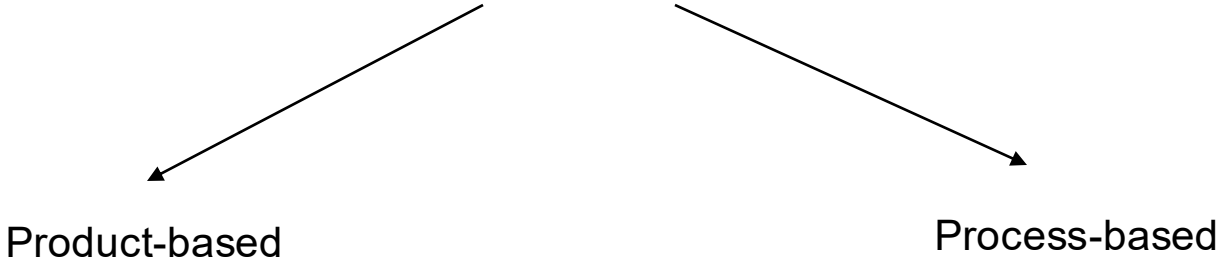
- The quality team should be independent from the development team so that they can take an objective view of the software.
  - This allows them to report on software quality without being influenced by software development issues.

Quality management and software development



Software Quality Management

- **Software quality management** is a management process, the goal of which is to develop and manage the quality of a software to make sure the product satisfies the user. (Wikipedia)
  - The goals of SQM (software quality management) is to make sure a product is in good levels of quality and meets the regulations and standards the customer needs from the product.



Software quality management

- Three principal concerns:
  - At the organizational level, quality management is concerned with establishing a framework of organizational processes and standards that will lead to high-quality software.
  - At the project level, quality management involves the application of specific quality processes and checking that these planned processes have been followed.
  - At the project level, quality management is also concerned with establishing a quality plan for a project. The quality plan should set out the quality goals for the project and define what processes and standards are to be used.

Quality planning

- A quality plan sets out the desired product qualities and how these are assessed and defines the most significant quality attributes.
- The quality plan should define the quality assessment process.
- It should set out which organisational standards should be applied and, where necessary, define new standards to be used.

- Quality plan structure:
  - Product introduction; Product plans; Process descriptions; Quality goals; Risks and risk management.
- Quality plans should be short, succinct documents
  - If they are too long, no-one will read them.

Product-based Quality:

Software quality attributes

Safety	Understandability	Portability
Security	Testability	Usability
Reliability	Adaptability	Reusability
Resilience	Modularity	Efficiency
Robustness	Complexity	Learnability

Quality conflicts

- It is not possible for any system to be optimized for all of these attributes – for example, improving robustness may lead to loss of performance.
- The quality plan should therefore define the most important quality attributes for the software that is being developed.
- The plan should also include a definition of the quality assessment process, an agreed way of assessing whether some quality, such as maintainability or robustness, is present in the product.

Example Scenario (Weigers)

- *"Hi, Phil, this is Maria again. I have a question about the new employee system you programmed. As you know, this system runs on our mainframe, and every department has to pay for its disk storage and CPU processing charges every month. It looks like the new system's files are using about twice as much disk space as the old system did. Even worse, the CPU charges are nearly three times what they used to be for a session. Can you tell me what's going on, please?"*

- *"Sure, Maria," said Phil. "Remember that you wanted this system to store much more data about each employee than the old system did, so naturally the database is much larger. Therefore, you're paying more for disk space usage each month. Also, you and the other product champions requested that the new system be much easier to use than the old one, so we designed that nice graphical user interface. However, the GUI consumes a lot more computer horsepower than the old system's simple character-mode display. So that's why your per-session processing charges are so much higher. The new system's a lot easier to use, isn't it?"*

Elaboration on Example Scenario

- Users naturally focus on specifying their functional requirements, but there is more to software success than just delivering the right functionality.

- Users also have the expectations about how well the product will work:
  - How easy it is to use,
  - How quickly it runs,
  - How often it fails,
  - How it handles exception conditions,
  - Etc.

Elaboration on Example Scenario (cont'd)

- Customers generally do not present their quality expectations explicitly.

- Questions that explore the customer's implicit expectations can lead to quality goal statements and design criteria that help the developers create a fully satisfactory product.

Difficulties with Quality Attributes

✧ Lack of awareness in the need of their specification

✧ Lack of know-how on;

- Categories and meanings,
- How to select and set relationships (trade-offs),
- How to specify,
- How to evaluate.

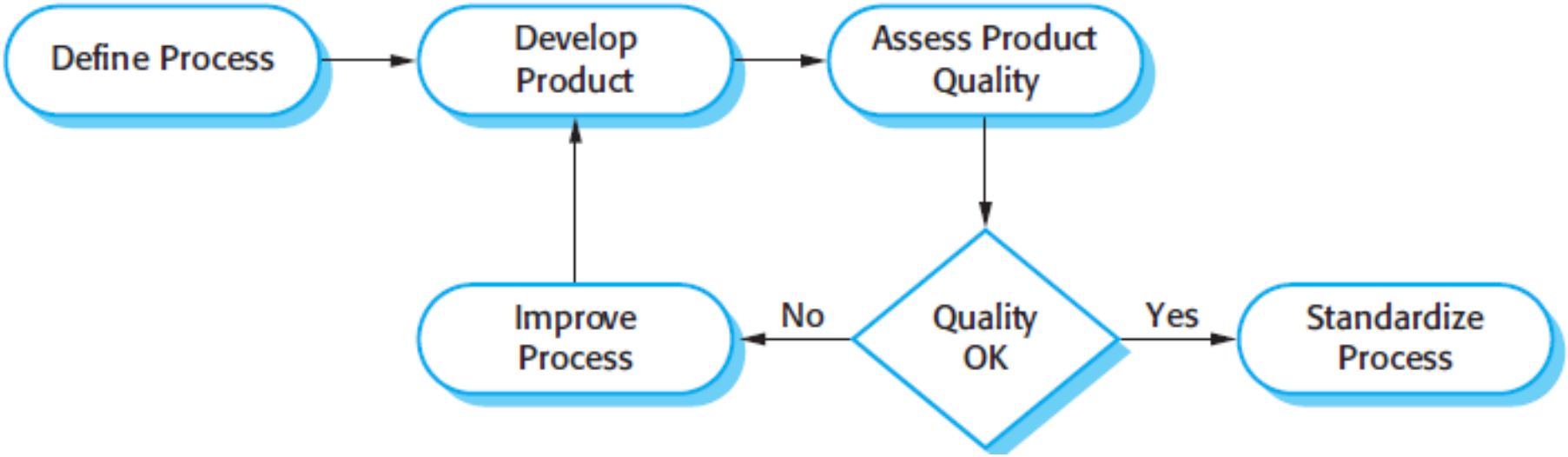
→ Without a precise specification of software quality, it is not possible to evaluate and achieve the quality of the sofware !!

Process and product quality

- **The quality of a developed product is influenced by the quality of the production process.**
  - This is important in software development as some product quality attributes are hard to assess.

- However, there is a very complex and poorly understood relationship between software processes and product quality.
  - The application of individual skills and experience is particularly important in software development;
  - External factors such as the novelty of an application or the need for an accelerated development schedule may impair product quality.

Process-based quality



Software standards

- Standards define the required attributes of a product or process. They play an important role in quality management.
- Standards may be international, national, organizational or project standards.
- Product standards define characteristics that all software components should exhibit.
  - e.g. a common programming style.
- Process standards define how the software process should be enacted.

Product and process standards

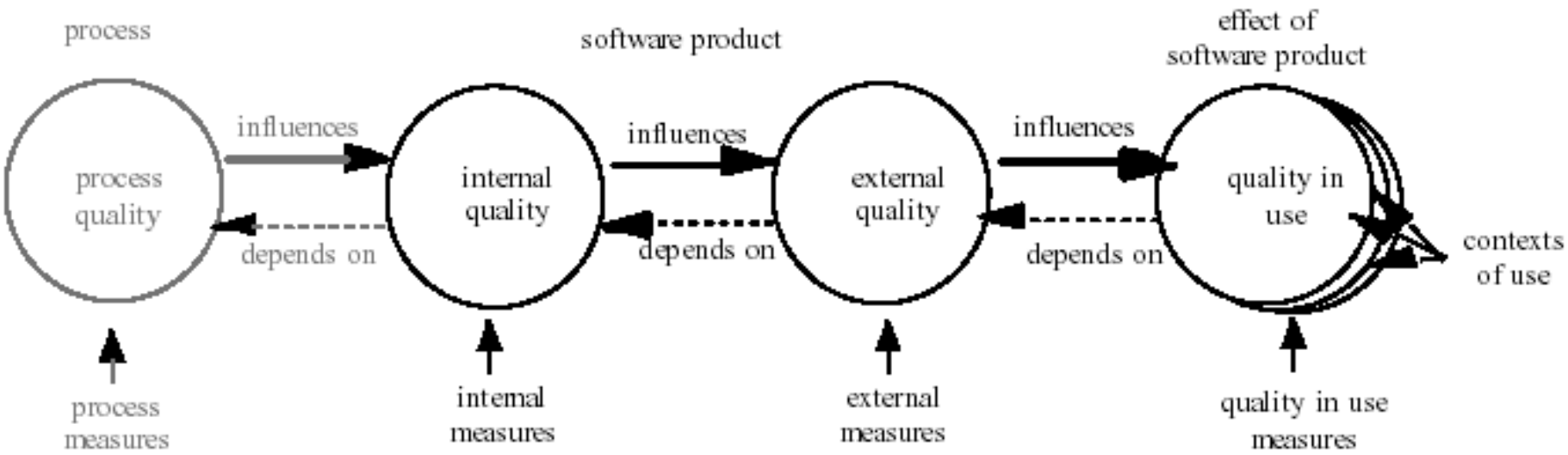
Product standards	Process standards
Design review form	Design review conduct
Requirements document structure	Submission of new code for system building
Method header format	Version release process
Java programming style	Project plan approval process
Project plan format	Change control process
Change request form	Test recording process

Product standards - example: Software Quality Attributes

- Augment the description of the product's functionality by describing the product's characteristics in various dimensions that are important either to users or developers.

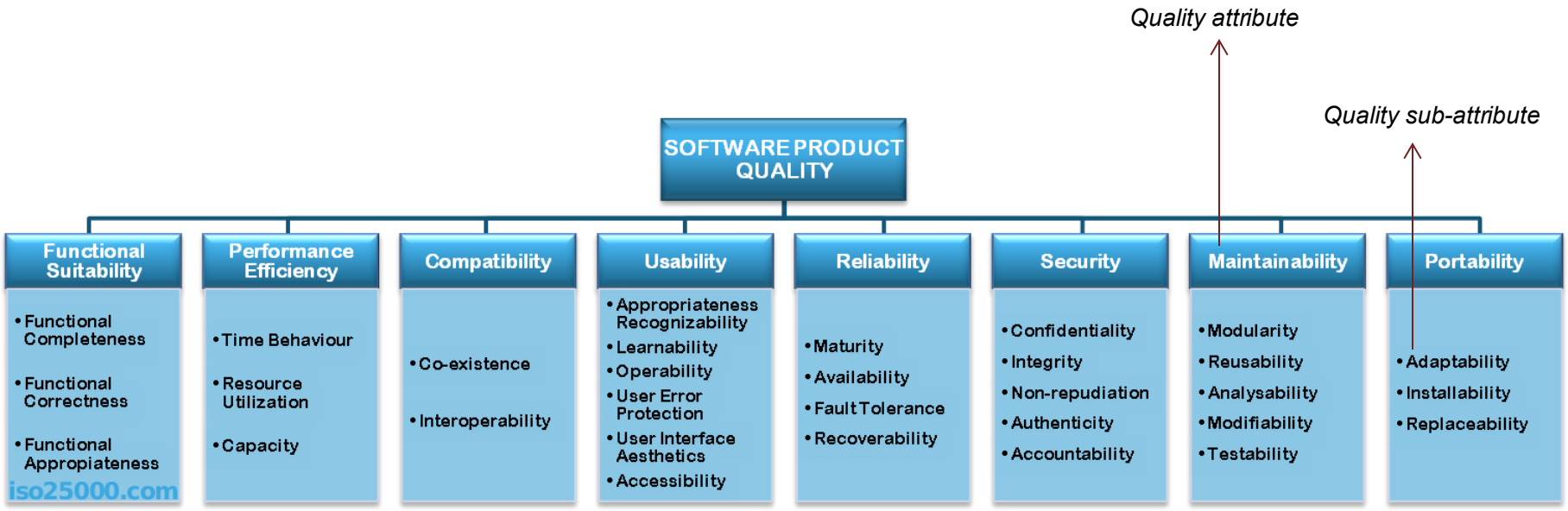
- E.g.;
  - Internal quality (developer perspective)
  - External quality (tester perspective)
  - Quality in use (user perspective)

ISO/IEC 25000 Quality Model Framework



- Process quality contributes to improving product quality, and product quality contributes to improving quality in use
- Evaluating quality in use can provide feedback to improve a product, and evaluating a product can provide feedback to improve a process

ISO 25010 Product Quality Attributes



Targets are set and results are evaluated via metrics defined for quality sub-attributes

Process standards - example: ISO 9001 standards framework

- An international set of standards that can be used as a basis for developing quality management systems.
- ISO 9001, the most general of these standards, applies to organizations that design, develop and maintain products, including software.
- It sets out general quality principles, describes quality processes in general and lays out the organizational standards and procedures that should be defined.
  - These should be documented in an organizational quality manual.

ISO 9001 core processes

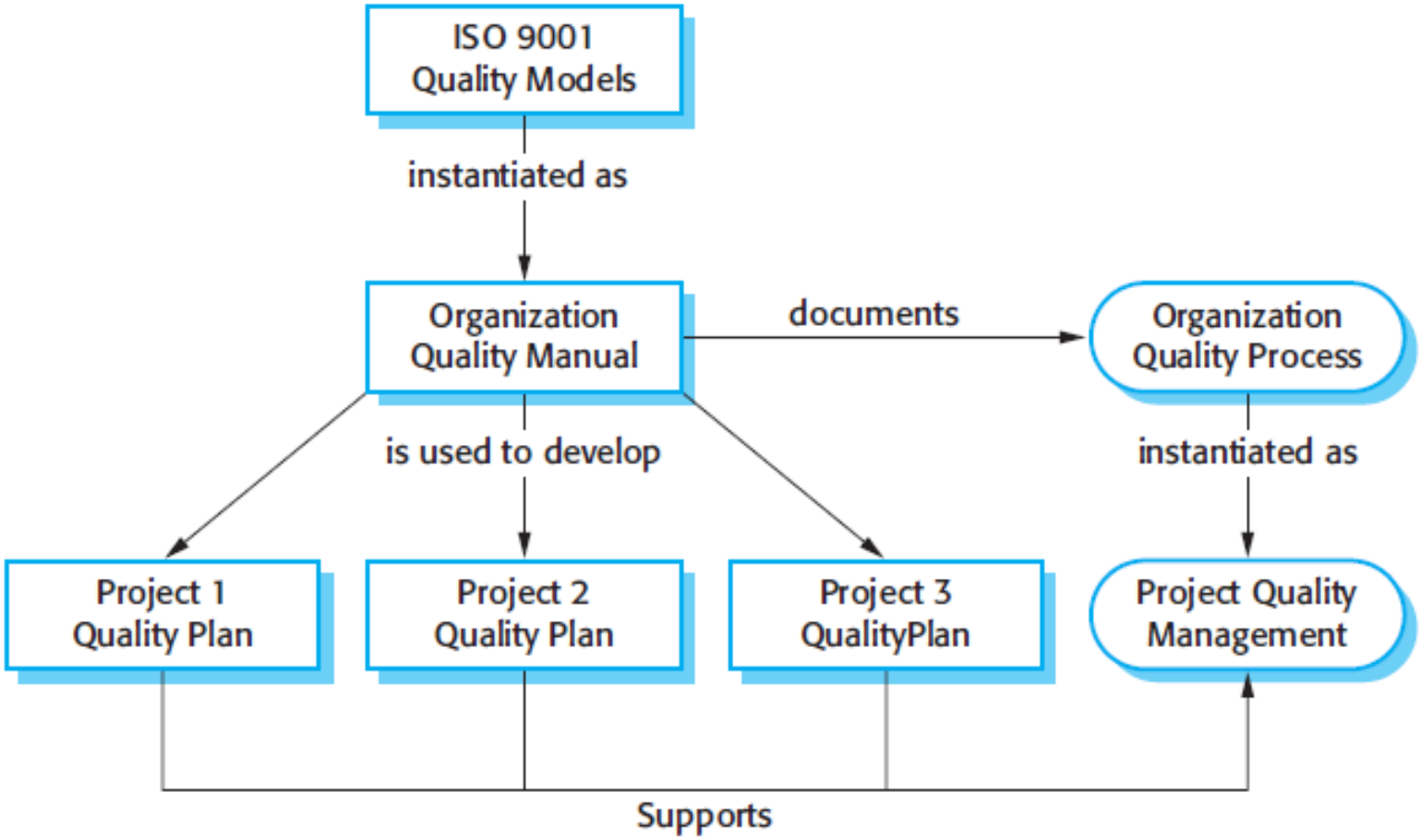
Product Delivery Processes

- Business Acquisition
- Design and Development
- Test
- Production and Delivery
- Service and Support

Supporting Processes

- Business Management
- Supplier Management
- Inventory Management
- Configuration Management

ISO 9001 and quality management



ISO 9001 certification

- Quality standards and procedures should be documented in an organisational quality manual.
- An external body may certify that an organisation's quality manual conforms to ISO 9000 standards.
- Some customers require suppliers to be ISO 9000 certified although the need for flexibility here is increasingly recognised.

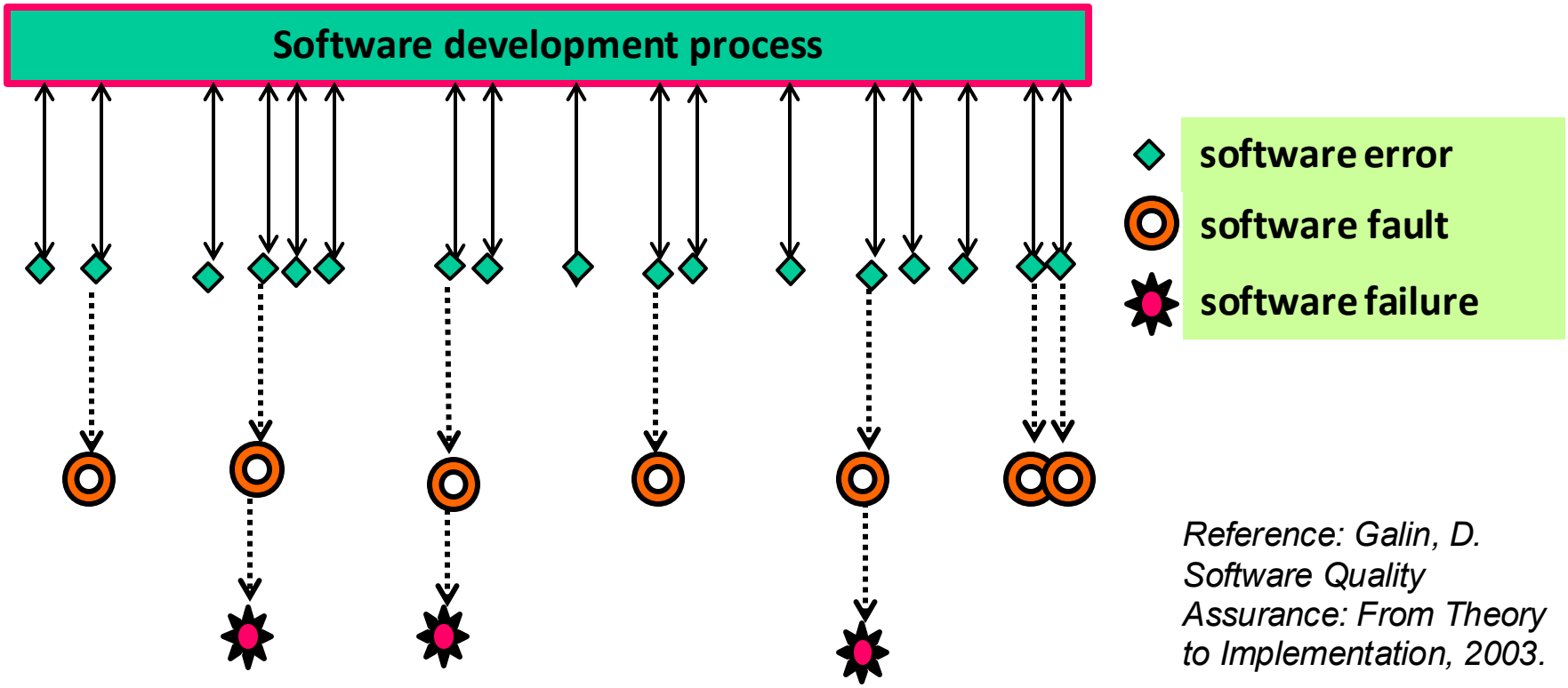
How to Achieve Quality?

- Traditional Approach: Defect detection
  - "Quality Control"
  - Achieving quality by targeting the product/service that is produced

- Modern Approach: Defect prevention
  - "Quality Assurance"
  - Achieving quality by targeting the system that produces the product/service

Software Errors, Faults and Failures

- *Bug/defect/fault* as a consequence of *human error*
  - results in non-conformance to requirements
  - may manifest as *failure* in running software



Reference: Galin, D. Software Quality Assurance: From Theory to Implementation, 2003.

When are defects introduced?

- The majority of defects are introduced in earlier phases.
  - Requirements are the top factor in a project's success or failure.

Phase	% of defects	% effort to fix
Requirements	56	82
Design	27	13
Code	7	1
Other	10	4

Cost of fixing defects

- Relative cost of fixing defects
  - benchmark: cost at requirements phase = 1

Phase found	Cost ratio
Requirements	1
Design	3 – 5
Coding	10
Unit / integration testing	15 - 40
System / acceptance testing	30 – 70
Production	40 - 1000

Quality Through Product/Service

- It is assumed that defects appear accidentally. When a defect is observed, the production line is <u>not</u> stopped, and defective parts are removed later.

- Forces workers to <u>not repeat the error</u>.
  - Workers may hide the problems.
  - Management may directly criticize workers.

Quality Through Process

- It is assumed that defects appear as a result of a problem in the production system. When a defect is detected, the production line is stopped. The reason of the defect is identified and removed (so that the same defect does not appear again).

- Forces workers to find defect and improve the production system.
  - It allows managers and workers to take responsibility of problems together.

Quality Assurance and Quality Control

- Quality Control
  - → For the purpose of defect detecting, at a certain time

- Quality Assurance
  - → For the purpose of defect prevention, over a period of time

- If we improve the quality of the development process, we both improve the product quality and reduce development time and cost.
  - "Quality is free"

Software Quality Assurance

- It is the cost-effective way of achieving quality during software development and maintenance.

- For the quality built into the software product, it aims to assure that processes applied and work-products produced within the software development life-cycle satisfy their specified requirements.

- It enables prevention and early detection of defects by planning, executing, and tracking a number of activites.

Scope of SQA Activities

- Software Quality Assurance (SQA) involves setting and executing mechanisms to ensure process and product quality throughout software development and maintenance.
  - Defining software processes, specifying standards to be used, setting up development environment, identfying SQA activities and their criteria
- Verification and Validation (V&V) involves assuring the quality of the work-products of the software development process against their specifications with the purpose of detecting non-conformances.
- Testing inovelves detecting defects by executing the code.

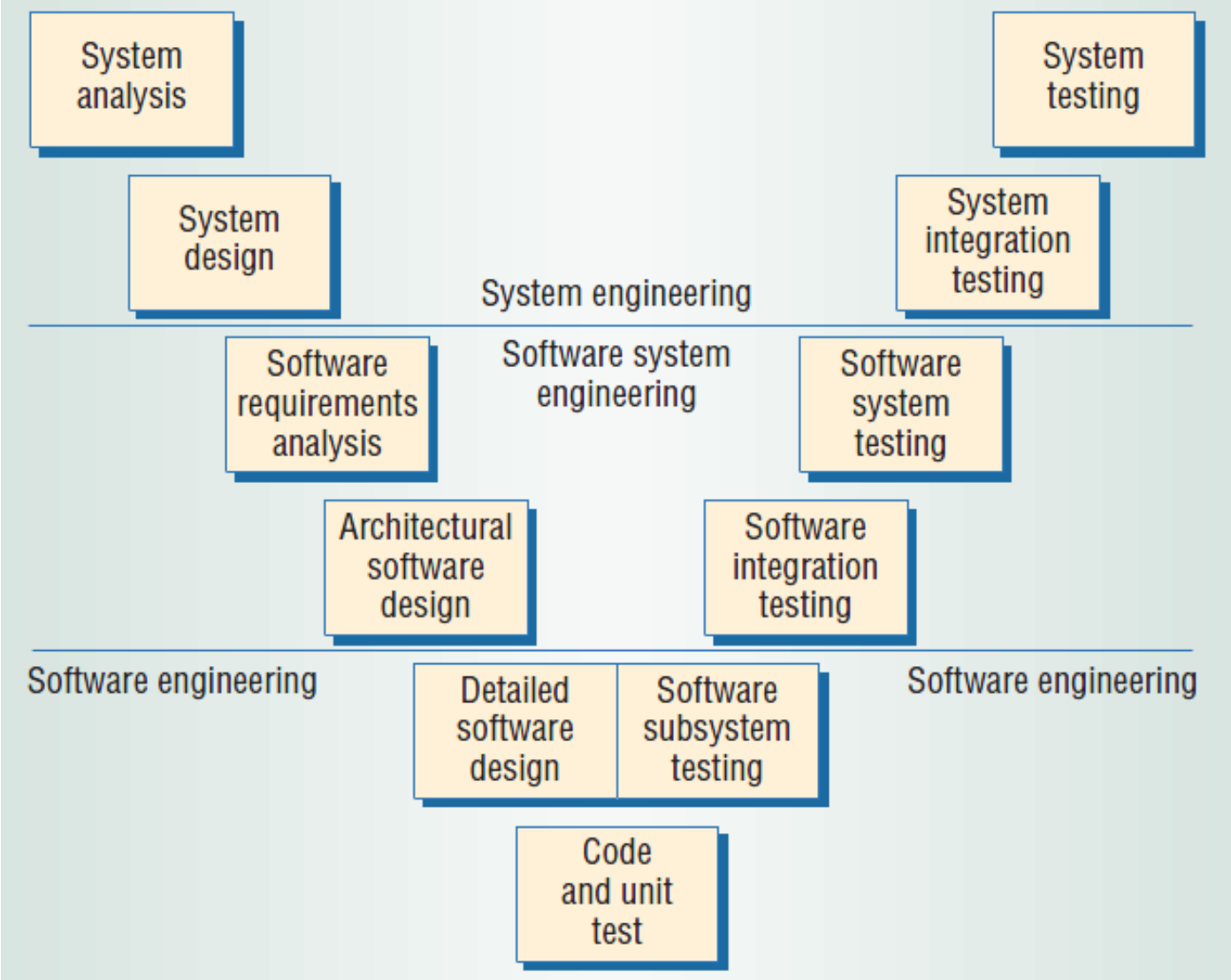
SQA includes V&V, and V&V includes Testing

SQA > V&V > Testing

Quality Through Product: Methods

- Static Methods: Applied by not executing the code (please refer to IEEE 1028:2008)
  - Management review
  - Technical review
  - Inspection
  - Walk-through
  - Audit
- Dynamic methods: Applied by executing the code
  - Unit testing
  - Integration testing
  - System testing
  - Acceptance testing

V – Model of Software Development

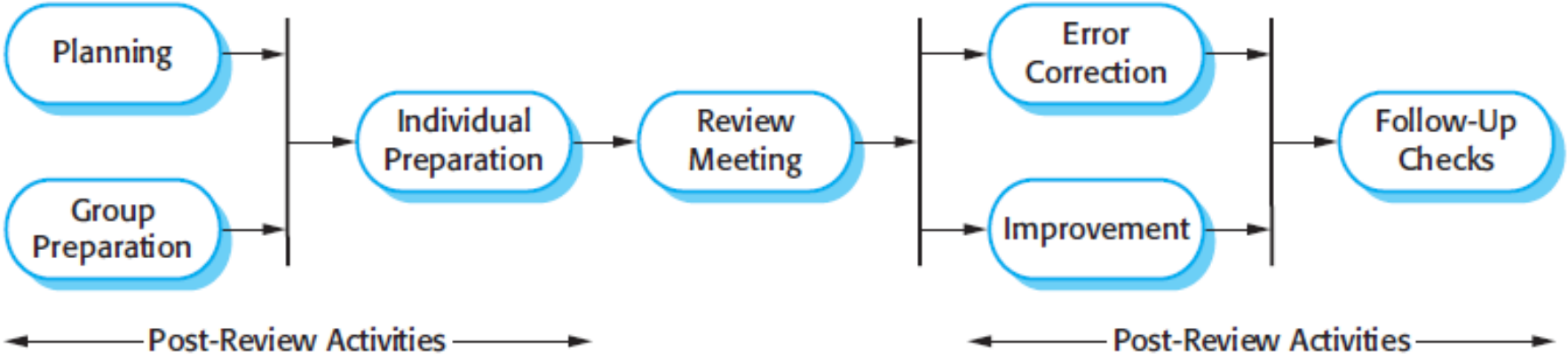


Reviews and inspections

- A group examines part or all of a process or system and its documentation to find potential problems.
- Software or documents may be 'signed off' at a review which signifies that progress to the next development stage has been approved by management.

- <u>There are different types of review with different objectives</u>
  - Inspections for defect removal (product);
  - Reviews for progress assessment (product and process);
  - Quality reviews (product and standards).

The software review process



Program inspections

- These are peer reviews where engineers examine the source of a system with the aim of discovering anomalies and defects.
- Inspections do not require execution of a system so may be used before implementation.
- They may be applied to any representation of the system (requirements, design, configuration data, test data, etc.).
- They have been shown to be an effective technique for discovering program errors.

Inspection checklists

- Checklist of common errors should be used to drive the inspection.
- Error checklists are programming language dependent and reflect the characteristic errors that are likely to arise in the language.
- Examples: Initialisation, constant naming, loop termination, array bounds, etc.

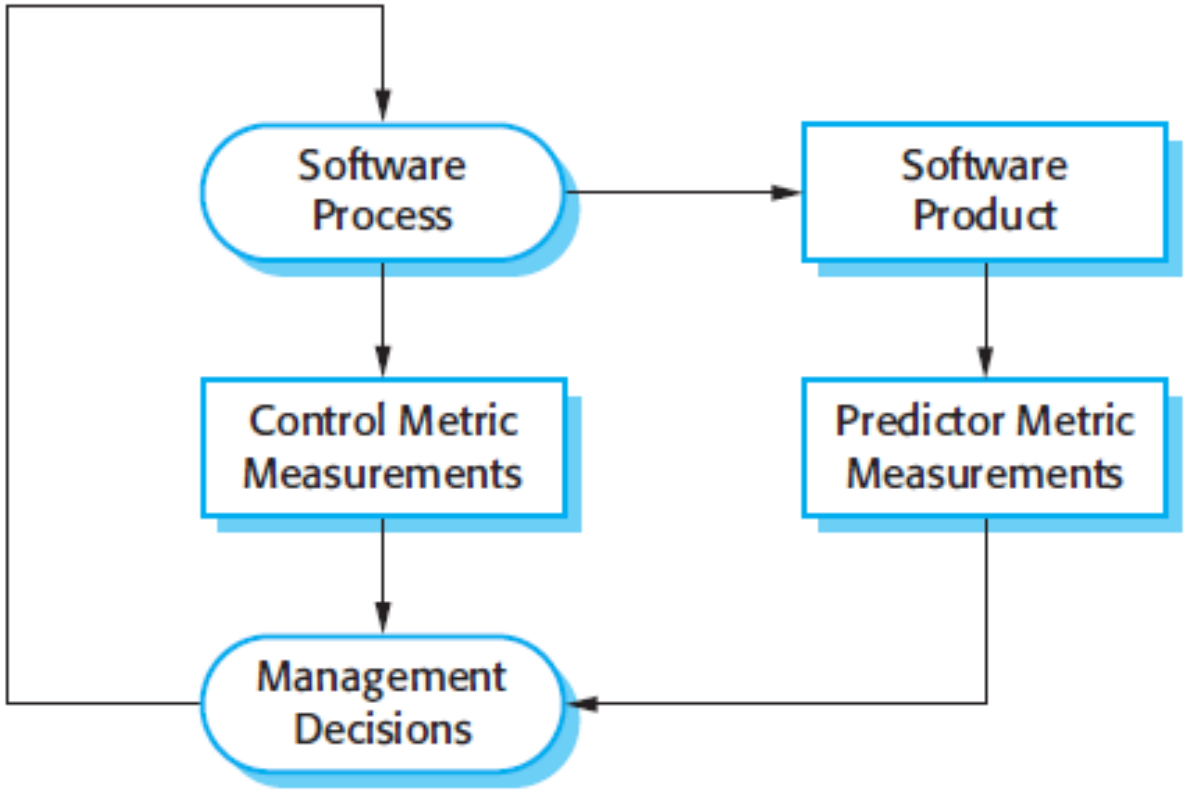
An inspection checklist (a)

Fault class	Inspection check
Data faults	• Are all program variables initialized before their values are used? • Have all constants been named? • Should the upper bound of arrays be equal to the size of the array or Size -1? • If character strings are used, is a delimiter explicitly assigned? • Is there any possibility of buffer overflow?
Control faults	• For each conditional statement, is the condition correct? • Is each loop certain to terminate? • Are compound statements correctly bracketed? • In case statements, are all possible cases accounted for? • If a break is required after each case in case statements, has it been included?
Input/output faults	• Are all input variables used? • Are all output variables assigned a value before they are output? • Can unexpected inputs cause corruption?

An inspection checklist (b)

Fault class	Inspection check
Interface faults	• Do all function and method calls have the correct number of parameters? • Do formal and actual parameter types match? • Are the parameters in the right order? • If components access shared memory, do they have the same model of the shared memory structure?
Storage management faults	• If a linked structure is modified, have all links been correctly reassigned? • If dynamic storage is used, has space been allocated correctly? • Is space explicitly deallocated after it is no longer required?
Exception management faults	• Have all possible error conditions been taken into account?

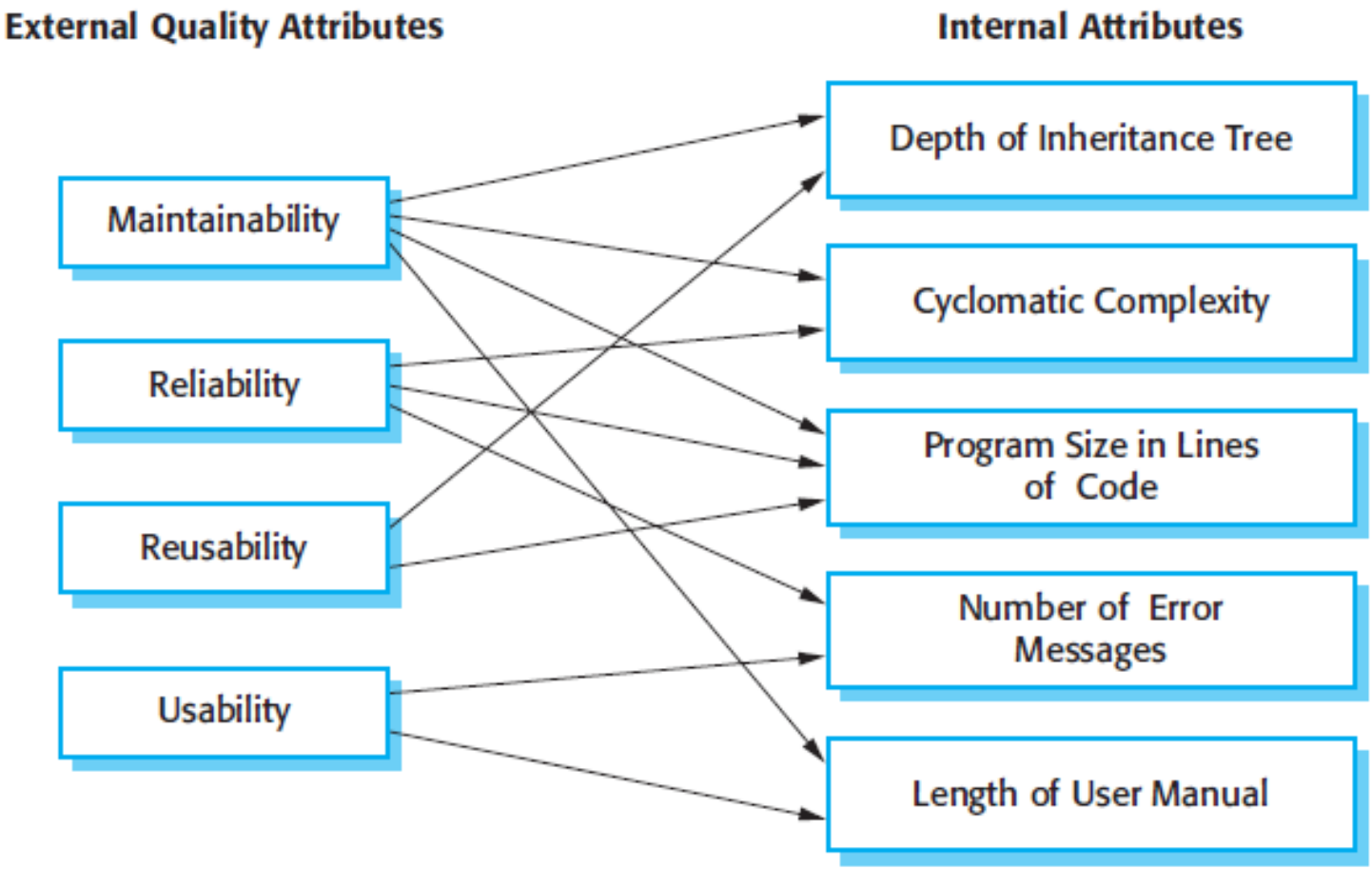
Predictor and control measurements



Software metric

- Any type of measurement which relates to a software system, process or related documentation
  - Lines of code in a program, the Fog index, the number of reported faults in a delivered software product, number of person-days required to develop a component
- Allow the software and the software process to be quantified.
- May be used to predict product attributes or to control the software process.
- Product metrics can be used for general predictions or to identify anomalous components.

Relationships between internal and external software quality attributes



Static software product metrics

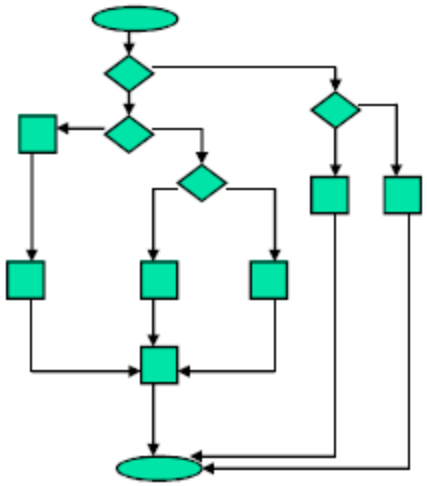
Software metric	Description
Fan-in/Fan-out	Fan-in is a measure of the number of functions or methods that call another function or method (say X). Fan-out is the number of functions that are called by function X. A high value for fan-in means that X is tightly coupled to the rest of the design and changes to X will have extensive knock-on effects. A high value for fan-out suggests that the overall complexity of X may be high because of the complexity of the control logic needed to coordinate the called components.
Length of code	This is a measure of the size of a program. Generally, the larger the size of the code of a component, the more complex and error-prone that component is likely to be. Length of code has been shown to be one of the most reliable metrics for predicting error-proneness in components.

Static software product metrics

Software metric	Description
Cyclomatic complexity	This is a measure of the control complexity of a program. This control complexity may be related to program understandability. I discuss cyclomatic complexity in Chapter 8.
Length of identifiers	This is a measure of the average length of identifiers (names for variables, classes, methods, etc.) in a program. The longer the identifiers, the more likely they are to be meaningful and hence the more understandable the program.
Depth of conditional nesting	This is a measure of the depth of nesting of if-statements in a program. Deeply nested if-statements are hard to understand and potentially error-prone.
Fog index	This is a measure of the average length of words and sentences in documents. The higher the value of a document's Fog index, the more difficult the document is to understand.

Some Examples

- Cyclomatic complexity



If G is the control flowgraph of program P and G has e edges and n nodes

$V(G) = e - n + 2$

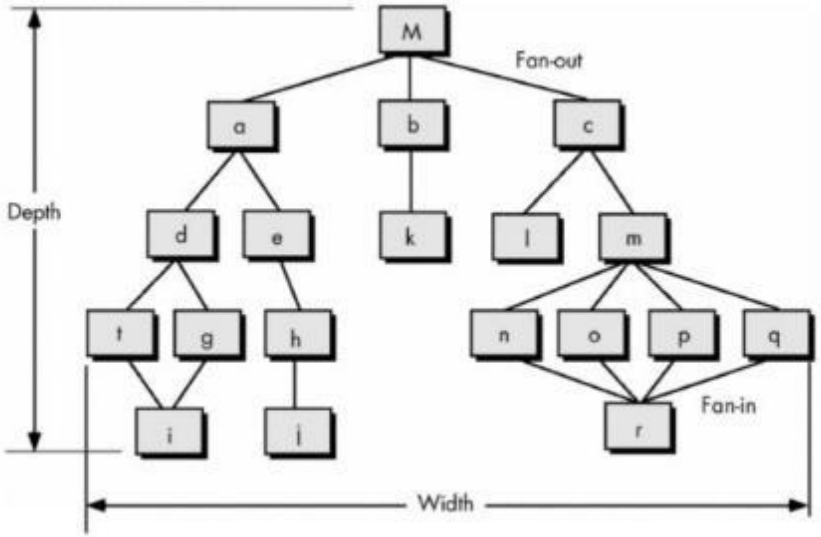
$V(G)$ is the number of linearly independent paths in G.

Here $e = 16$, $n = 13$, $V(G) = 5$

More simply, if d is the number of decision nodes in G than

$V(G) = d + 1$

- Fan-in & Fan-out



The CK object-oriented metrics suite

Object-oriented metric	Description
Weighted methods per class (WMC)	This is the number of methods in each class, weighted by the complexity of each method. Therefore, a simple method may have a complexity of 1, and a large and complex method a much higher value. The larger the value for this metric, the more complex the object class. Complex objects are more likely to be difficult to understand. They may not be logically cohesive, so cannot be reused effectively as superclasses in an inheritance tree.
Depth of inheritance tree (DIT)	This represents the number of discrete levels in the inheritance tree where subclasses inherit attributes and operations (methods) from superclasses. The deeper the inheritance tree, the more complex the design. Many object classes may have to be understood to understand the object classes at the leaves of the tree.
Number of children (NOC)	This is a measure of the number of immediate subclasses in a class. It measures the breadth of a class hierarchy, whereas DIT measures its depth. A high value for NOC may indicate greater reuse. It may mean that more effort should be made in validating base classes because of the number of subclasses that depend on them.

The CK object-oriented metrics suite

Object-oriented metric	Description
Coupling between object classes (CBO)	Classes are coupled when methods in one class use methods or instance variables defined in a different class. CBO is a measure of how much coupling exists. A high value for CBO means that classes are highly dependent, and therefore it is more likely that changing one class will affect other classes in the program.
Response for a class (RFC)	RFC is a measure of the number of methods that could potentially be executed in response to a message received by an object of that class. Again, RFC is related to complexity. The higher the value for RFC, the more complex a class and hence the more likely it is that it will include errors.
Lack of cohesion in methods (LCOM)	LCOM is calculated by considering pairs of methods in a class. LCOM is the difference between the number of method pairs without shared attributes and the number of method pairs with shared attributes. The value of this metric has been widely debated and it exists in several variations. It is not clear if it really adds any additional, useful information over and above that provided by other metrics.

Software component analysis

- System component can be analyzed separately using a range of metrics.
- The values of these metrics may then compared for different components and, perhaps, with historical measurement data collected on previous projects.
- Anomalous measurements, which deviate significantly from the norm, may imply that there are problems with the quality of these components.

FOR STUDENT REVIEW…

Scope of quality management

- Quality management is particularly important for large, complex systems. The quality documentation is a record of progress and supports continuity of development as the development team changes.
- For smaller systems, quality management needs less documentation and should focus on establishing a quality culture.

Software quality

- Quality, simplistically, means that a product should meet its specification.

- This is problematical for software systems
  - There is a tension between customer quality requirements (efficiency, reliability, etc.) and developer quality requirements (maintainability, reusability, etc.);
  - Some quality requirements are difficult to specify in an unambiguous way;
  - Software specifications are usually incomplete and often inconsistent.

Software fitness for purpose

- The focus may be ‘fitness for purpose’ rather than specification conformance.
  - Have programming and documentation standards been followed in the development process?
  - Has the software been properly tested?
  - Is the software sufficiently dependable to be put into use?
  - Is the performance of the software acceptable for normal use?
  - Is the software usable?
  - Is the software well-structured and understandable?

Importance of standards

- Encapsulation of best practice - avoids repetition of past mistakes.
- They are a framework for defining what quality means in a particular setting i.e. that organization's view of quality.
- They provide continuity - new staff can understand the organisation by understanding the standards that are used.

Problems with standards

- They may not be seen as relevant and up-to-date by software engineers.
- They often involve too much bureaucratic form filling.
- If they are unsupported by software tools, tedious form filling work is often involved to maintain the documentation associated with the standards.

Standards development

- Involve practitioners in development. Engineers should understand the rationale underlying a standard.
- Review standards and their usage regularly. Standards can quickly become outdated and this reduces their credibility amongst practitioners.
- Detailed standards should have specialized tool support. Excessive clerical work is the most significant complaint against standards.
  - Web-based forms are not good enough.

Quality reviews

- A group of people carefully examine part or all of a software system and its associated documentation.
- Code, designs, specifications, test plans, standards, etc. can all be reviewed.
- Software or documents may be 'signed off' at a review which signifies that progress to the next development stage has been approved by management.

Reviews and agile methods

- The review process in agile software development is usually informal.
  - In Scrum, for example, there is a review meeting after each iteration of the software has been completed (a sprint review), where quality issues and problems may be discussed.
- In extreme programming, pair programming ensures that code is constantly being examined and reviewed by another team member.
- XP relies on individuals taking the initiative to improve and re-factor code. Agile approaches are not usually standards-driven, so issues of standards compliance are not usually considered.

Agile methods and inspections

- Agile processes rarely use formal inspection or peer review processes.
- Rather, they rely on team members cooperating to check each other's code, and informal guidelines, such as 'check before check-in', which suggest that programmers should check their own code.
- Extreme programming practitioners argue that pair programming is an effective substitute for inspection as this is, in effect, a continual inspection process.
- Two people look at every line of code and check it before it is accepted.

Software measurement and metrics

- Software measurement is concerned with deriving a numeric value for an attribute of a software product or process.
- This allows for objective comparisons between techniques and processes.
- Although some companies have introduced measurement programs, most organisations still don’t make systematic use of software measurement.
- There are few established standards in this area.

Use of measurements

- To assign a value to system quality attributes
  - By measuring the characteristics of system components, such as their cyclomatic complexity, and then aggregating these measurements, you can assess system quality attributes, such as maintainability.
- To identify the system components whose quality is sub-standard
  - Measurements can identify individual components with characteristics that deviate from the norm. For example, you can measure components to discover those with the highest complexity. These are most likely to contain bugs because the complexity makes them harder to understand.

Metrics assumptions

- A software property can be measured.
- The relationship exists between what we can measure and what we want to know. We can only measure internal attributes but are often more interested in external software attributes.
- This relationship has been formalised and validated.
- It may be difficult to relate what can be measured to desirable external quality attributes.

Product metrics

- A quality metric should be a predictor of product quality.

- Classes of product metric
  - Dynamic metrics which are collected by measurements made of a program in execution;
    - Dynamic metrics help assess efficiency and reliability
  - Static metrics which are collected by measurements made of the system representations;
    - Static metrics help assess complexity, understandability, and maintainability.

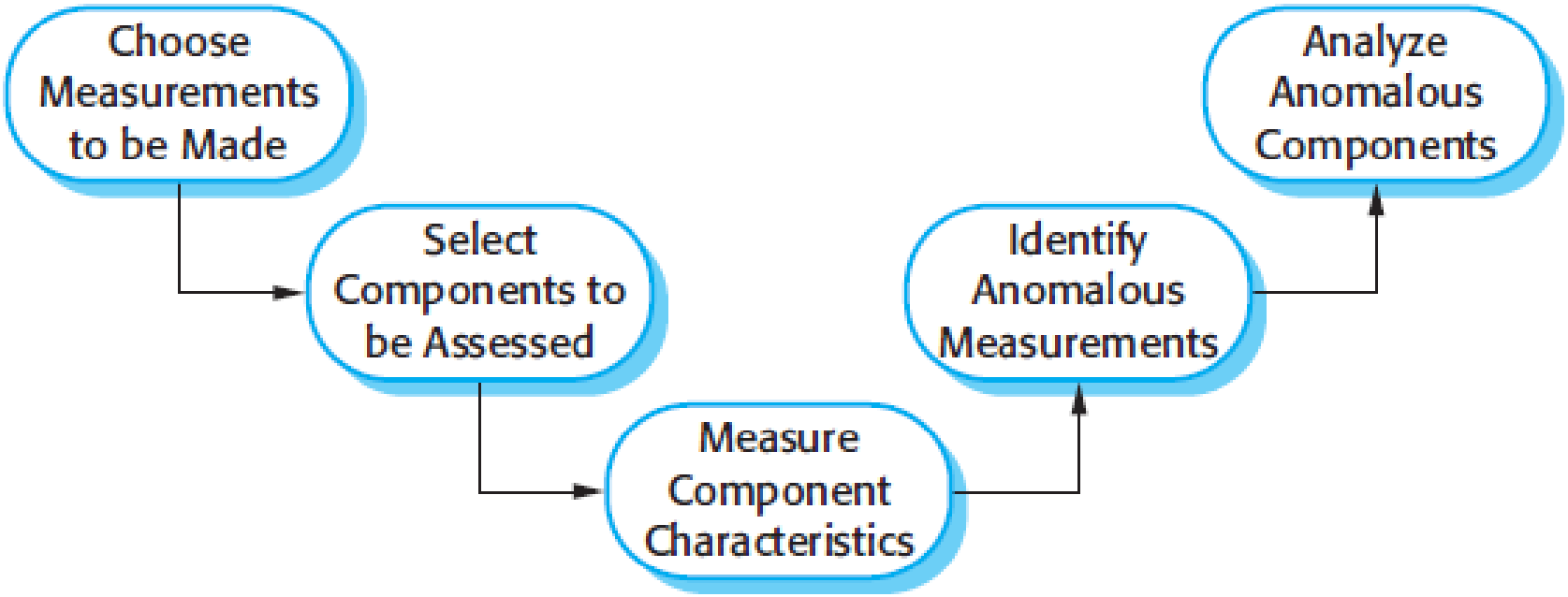
Dynamic and static product metrics

- Dynamic metrics are closely related to software quality attributes
  - It is relatively easy to measure the response time of a system (performance attribute) or the number of failures (reliability attribute).
- Static metrics have an indirect relationship with quality attributes
  - You need to try and derive a relationship between these metrics and properties such as complexity, understandability and maintainability.

Problems with measurement in industry

- It is impossible to quantify the return on investment of introducing an organizational metrics program.
- There are no standards for software metrics or standardized processes for measurement and analysis.
- In many companies, software processes are not standardized and are poorly defined and controlled.
- Most work on software measurement has focused on code-based metrics and plan-driven development processes. However, more and more software is now developed by configuring ERP systems or COTS.
- Introducing measurement adds additional overhead to processes.

The process of product measurement



Measurement surprises

- Reducing the number of faults in a program leads to an increased number of help desk calls
  - The program is now thought of as more reliable and so has a wider more diverse market. The percentage of users who call the help desk may have decreased but the total may increase;
  - A more reliable system is used in a different way from a system where users work around the faults. This leads to more help desk calls.

Key points

- Software quality management is concerned with ensuring that software has a low number of defects and that it reaches the required standards of maintainability, reliability, portability and so on.
- SQM includes defining standards for processes and products and establishing processes to check that these standards have been followed.
- Software standards are important for quality assurance as they represent an identification of ‘best practice’.
- Quality management procedures may be documented in an organizational quality manual, based on the generic model for a quality manual suggested in the ISO 9001 standard.

Key points

- Reviews of the software process deliverables involve a team of people who check that quality standards are being followed.
- In a program inspection or peer review, a small team systematically checks the code. They read the code in detail and look for possible errors and omissions
- Software measurement can be used to gather data about software and software processes.
- Product quality metrics are particularly useful for highlighting anomalous components that may have quality problems.